

1.

What is the l-value of a variable?

What is the r-value?

(2 marks)

The address of a variable is sometimes called its **l-value**, because the address is what is required when the name of a variable appears in the left side of an assignment.

A variable's value is sometimes called its **r-value** because it is what is required when the name of the variable appears in the right side of an assignment statement. To access the *r*-value, the *l*-value must be determined first. Such determinations are not always simple. For example, scoping rules can greatly complicate matters, as is discussed in Section 5.5.

2.

What are the primary tasks of a lexical analyzer? (3 points)

[describe it with your own words, what steps are performed and what produced by it]

A lexical analyzer is essentially a pattern matcher. A pattern matcher attempts to find a substring of a given string of characters that matches a given character pattern. Pattern matching is a traditional part of computing. One of the earliest uses of pattern matching was with text editors, such as the ed line editor, which was introduced in an early version of UNIX. Since then, pattern matching has found its way into some programming languages—for example, Perl and JavaScript. It is also available through the standard class libraries of Java, C++, and C#.

A lexical analyzer serves as the front end of a syntax analyzer. Technically, lexical analysis is a part of syntax analysis. A lexical analyzer performs syntax analysis at the lowest level of program structure. An input program appears to a compiler as a single string of characters. The lexical analyzer collects characters into logical groupings and assigns internal codes to the groupings according to their structure. In Chapter 3, these logical groupings are named lexemes, and the internal codes for categories of these groupings are named tokens. Lexemes are recognized by matching the input character string against character string patterns. Although tokens are usually represented as integer values, for the sake of readability of lexical and syntax analyzers, they are often referenced through named constants.

Lexical analyzers extract lexemes from a given input string and produce the corresponding tokens. In the early days of compilers, lexical analyzers often processed an entire source program file and produced a file of tokens and lexemes. Now, however, most lexical analyzers are subprograms that locate the next lexeme in the input, determine its associated token code, and return them to the caller, which is the syntax analyzer. So, each call to the lexical analyzer

returns a single lexeme and its token. The only view of the input program seen by the syntax analyzer is the output of the lexical analyzer, one token at a time. The lexical-analysis process includes skipping comments and white space outside lexemes, as they are not relevant to the meaning of the program. Also, the lexical analyzer inserts lexemes for user-defined names into the symbol table, which is used by later phases of the compiler. Finally, lexical analyzers detect syntactic errors in tokens, such as ill-formed floating-point literals, and report such errors to the user.

Purpose of Lexical Analyzer z Main task: to read input characters and group them into "tokens."
z Secondary tasks: z Skip comments and whitespace; z Correlate error messages with source program (e.g., line number of error).

3.

Explain how the coercion rules of a language affect its error detection. (4 marks)

The coercion also results in a loss of one of the benefits of strong typing—error detection. For example, suppose a program had the **int** variables *a* and *b* and the **float** variable *d*. Now, if a programmer meant to type *a + b*, but mistakenly typed *a + d*, the error would not be detected by the compiler. The value of *a* would simply be coerced to **float**. So, the value of strong typing is weakened by coercion. Languages with a great deal of coercion, like C, and C++, are less reliable than those with no coercion, such as ML and F#. Java and C# have half as many assignment type coercions as C++, so their error detection is better than that of C++, but still not nearly as effective as that of ML and F#. The issue of coercion is examined in detail in Chapter 7.

4.

What is dynamic scoping and a dynamic ancestor of a subprogram?

Show an example! (2 points)

Dynamic scoping is based on the calling sequence of subprograms, not on their spatial relationship to each other. Thus, the scope can be determined only at run time.

- Dynamic parent of a subprogram is the subprogram that call him. The dynamic parent of subprogram such as *a*, and its dynamic parent, and so forth up to the including the main, are called the dynamic ancestors of *a*.

Procedure main()

```

Var x : integer
Procedure ali
Begin
  ...
end
Procedure ahmed
begin
  .....Ali().....
end
End
Begin main
  ...Ahmed();
End
Dynamic ancestors of Ahmed are : main
Dynamic ancestors of Ali are : Ahmed and main

```

5.

What is a coroutine? (2 marks)

A **coroutine** is a special kind of subprogram. Rather than the master-slave relationship between a caller and a called subprogram that exists with conventional subprograms, caller and called coroutines are more equitable. In fact, the coroutine control mechanism is often called the **symmetric unit control model**.

6.

Consider the following C program, which variable has the longest scope?

Explain your answer with details! [3 marks]

```

int a;
int main()
{
    int b;
    // ..
    // ..
}
int c;

```

a is accessible everywhere.

b is limited to main()

c is accessible only after its declaration.

7.

Consider the program below in a hypothetical language which allows global variable and a choice of call by reference or call by value methods of parameter passing.

What will be the output of the program for the two parameter passing mechanisms?

Explain it for pass-by-value and pass-by-reference as well! [4 marks]

```
int i ;
program main ()
{
    int j = 60;
    i = 50;
    call f (i, j);
    print i, j;
}
procedure f (x, y)
{
    i = 100;
    x = 10;
    y = y + i ;
}
```

CALL BY VALUE :- i as global variable declared. Then in $main()$ a local variable j as integer declared i.e $j = 60$ And global variable i initialized to 50 by $i = 50$. Now procedure f called and values of i and j are passed to it. i.e., in $f(i, j) \rightarrow f(x, y)$ content of memory location of i (here 50) is copied to memory location of x (which is different from i) and content of memory location of j (here, 60) is copied to memory location of y . Then in $f(x, y)$ $i = 100$ changes the global i to 100, $X = 10$ changes the local X from 50 to 10 and $Y = y + i$ means $y = 60 + 100 = 160$. Now when return back to main, i and j will be 100 and 60 respectively.

CALL BY REFERENCE:- Now procedure f called and passed reference of i and j to it. i.e., in $f(i, j) \rightarrow f(x, y)$ x and y are new names (aliases) pointing to the same memory location of i and j respectively. So, $i = 100$ changes the global i to 100 and $x = 10$ means x as well as global $i = 10$ (as the i being passed is the global variable and x and i share the same address).
 $y = y + i$ means $y = 60 + 10 = 70$ and this changes the value of j also to 70 as j and y have the same address. Now when return back to main, i and j will be 10 and 70 respectively.

8.

Explain why dynamic type bindings is closely related to implicit heap-dynamic variables, using the following code example:

```
var a;  
....  
a = "Hello";  
....  
a = 123.456;
```

Variable `a` gets its type at runtime, i.e. dynamic type binding, when a value is assigned to it. Since variable `a` may often change the types of values it holds, hence the memory locations where the values are stored, `a` must be a reference variable and its dynamically assigned value must be in a dynamically allocated memory location, i.e. the heap.

9.

What is the Pass-by-Reference model of parameter passing? [describe it]
Explain in your answer the parameter collision problem!
(4 points)

Pass by reference (also called **pass by address**) means to **pass** the **reference** of an argument in the calling function to the corresponding formal **parameter** of the called function so that a copy of the address of the actual **parameter** is made in memory, i.e. the caller and the callee use the same **variable** for the **parameter**.

Parameter collision problem occurs in pass-by-result model of parameter passing. When we have the same actual parameters in the calling function and different formal parameters in the called function, the order in which the actual parameters are copied determines the value.

10.

What is an overloaded subprogram? (2 marks)

Overloaded subprogram refers to a subprogram that has the same name as other subprograms but has different number, type, order of parameters or return value in the same programming circumstance.

11.

Write a simple assignment statement with one arithmetic operator in some language you know. For each component of the statement, list the various bindings that are required to determine the semantics when the statement is executed. For each binding, indicate the binding time used for the language. (4 marks)

```
count = count + 5;
```

Some of the bindings and their binding times for the parts of this assignment statement are as follows:

- The type of `count` is bound at compile time.
- The set of possible values of `count` is bound at compiler design time.
- The meaning of the operator symbol `+` is bound at compile time, when the types of its operands have been determined.
- The internal representation of the literal `5` is bound at compiler design time.
- The value of `count` is bound at execution time with this statement.

(C++)

```
int count;
```

```
count = count + 5;
```

Possible types for count: set at language design time. **Type of count:** bound at compile time.

Set of possible values of count: bound at compiler design time. **Value of count:** bound at execution time with this statement. **Set of possible meanings for the operator symbol `+`:** bound at language definition time. **Meaning of the operator symbol `+` in this statement:** bound at compile time.

Internal representation of the literal `5`: bound at compiler design time.

12. Type Equivalence

Two types are **equivalent** if an operand of one type in an expression is substituted for one of the other type, without coercion. Type equivalence is a strict form of type compatibility—compatibility without coercion.